Защита REST API и валидация пользовательских данных

Понятно о проверке данных, express-validator, Helmet и rate limiting • Internet Shop

Зачем нужна эта лекция

- Даже правильно работающий frontend не гарантирует, что сервер получит корректные данные.
- Научимся проверять запросы до обращения к Prisma и PostgreSQL.
- Добавим простые меры защиты от слишком больших и слишком частых запросов.

Учебные вопросы

№	Учебный вопрос	Главная задача
1	Почему нельзя доверять входным данным	Понять, что и где проверять
2	Проверка данных через express-validator	Создать понятные правила и сообщения об ошибках
3	Безопасная передача данных в Prisma	Не допускать неожиданных полей
4	Ограничение размера запросов и Helmet	Защитить обработку JSON и HTTP-ответы
5	Ограничение частоты запросов	Настроить rate limiting и собрать защиты вместе

Исходный проект: Internet Shop

- Express обрабатывает запросы к /api/products и /api/auth/login.
- Prisma передаёт данные в PostgreSQL.
- JWT и requireRole("admin") уже проверяют личность и полномочия.
- Теперь дополнительно проверим сами данные запроса и усилим HTTP-защиту.

Учебный вопрос 1. Почему нельзя доверять данным

- Разберём, какие данные может отправить клиент и почему frontend-проверок недостаточно.

Что такое пользовательский ввод

- Пользовательский ввод — любые данные, которые клиент передаёт серверу.
- req.body — данные JSON, например название и цена товара.
- req.params — часть URL, например /api/products/12.
- req.query — параметры после ?, например /api/products?sort=price.
- Эти значения можно изменять даже без нашей HTML-формы.

Пример: что пользователь может прислать

Ожидали	Клиент прислал	Проблема
price: 2500	price: -10	Отрицательная цена
name: "Мышь"	name: ""	Пустое название
/products/5	/products/abc	Неверный ID
role: "user"	role: "admin"	Попытка изменить права

Что такое валидация

- Валидация (validation) — проверка входных данных по заранее заданным правилам.
- Например: цена — положительное целое число; название — от 2 до 150 символов.
- Если правило нарушено, API возвращает ошибку, а не записывает данные в базу.
- Валидация не заменяет авторизацию: право создавать товар проверяется отдельно.

Frontend и backend: где проверять данные

- Frontend проверяет данные ради удобства: быстро показывает подсказку пользователю.
- Backend проверяет фактически полученный HTTP-запрос — это обязательная проверка.
- Злоумышленник может обойти HTML-форму и отправить запрос через curl или Postman.
- Поэтому валидация на сервере нужна даже при идеальной frontend-форме.

Недоверенные данные → валидация → бизнес-правила → база данных

HTTP-запрос
(недоверенные данные от пользователя)

req.body
Недоверенные данные

req.params
Недоверенные данные

req.query
Недоверенные данные



Пользователь может отправить любые данные!

Server-side validation

- тип
- формат
- длина
- диапазон
- allowlist

Некорректные данные

400 Bad Request

Ошибка валидации (некорректный формат, недопустимые значения и т.п.)

Service rules

- бизнес-правила
- наличие товара
- права доступа
- остаток

Нарушены бизнес-правила

400 Bad Request

Бизнес-правило не выполнено (нет товара, недостаточно прав, недостаточный остаток и т.п.)

Prisma

ORM / Data Access

- безопасные запросы
- типизация
- защита от SQL-инъекций
- работа с моделями

PostgreSQL

- хранение данных
- транзавции
- целостность
- надежность

Два вида проверок: форма и смысл

- Синтаксическая проверка: правильно ли записано значение? Например price — целое число.
- Смысловая проверка: допустимо ли значение для бизнеса? Например товара достаточно на складе.
- Проверку формата можно выполнять middleware.
- Проверку наличия товара и остатка обычно выполняет Service с запросом к БД.

Правила для товара Internet Shop

Поле	Правило	Пример ошибки
name	Строка от 2 до 150 символов	Пустое имя
price	Положительное целое число	-200
categoryId	Положительный целочисленный ID	"abc"
inStock	Логическое значение	"maybe"
id в URL	Положительное целое число	/products/test

Учебный вопрос 2. express-validator

- Сначала познакомимся с библиотекой, затем напишем правила для создания товара.

Что такое express-validator

- express-validator — библиотека для проверки и преобразования данных HTTP-запросов в Express.
- Она позволяет записывать правила в виде цепочек: поле → проверка → сообщение.
- Проверяет req.body, req.params, req.query и другие части запроса.
- При ошибке правила собирают сведения о проблемах; ответ клиенту мы формируем сами.

Зачем использовать express-validator

- Без библиотеки пришлось бы писать множество ручных if для каждого поля.
- С ней правила можно вынести в отдельный файл и повторно использовать.
- Обработчики маршрутов становятся короче и понятнее.
- Пример: перед созданием Product проверить name и price и вернуть HTTP 400 при ошибке.

Установка и подключение express-validator

BASH

```
npm install express-validator
```

Что важно

- Команда устанавливает npm-пакет в серверный проект.
- Запускайте в каталоге проекта рядом с package.json.
- Сначала разберём простой пример, затем подключим правила к маршруту.

Первая проверка: название товара

```javascript
import { body } from "express-validator";

const checkName = [
  body("name")
    .isString()
    .isLength({ min: 2, max: 150 })
];
```

Что важно

- body() – это функция предназначенная для проверки полей тела HTTP-запроса.
- body("name") выбирает поле name из JSON-тела.
- isString() проверяет тип поля.
- isLength() проверяет длину.
- Правило не записывает товар в базу — оно только проверяет запрос.

Основные инструменты express-validator

Функция	Для чего	Пример
body("name")	Проверить поле JSON	POST /api/products
param("id")	Проверить параметр URL	GET /api/products/5
query("sort")	Проверить параметр после ?	GET /api/products?sort=price
validationResult(req)	Получить список ошибок	HTTP 400
matchedData(req)	Взять поля, прошедшие проверку	Данные для Prisma

Validation flow in Express: body / param / query → validationResult

Данные из разных источников (req.body, req.params, req.query) проходят через цепочки проверок **express-validator**.
Результат проверяется с помощью validationResult(req).

Internet Shop
Learn. Build. Deploy.

1. Источники данных запроса

req.body
Тело запроса (JSON)

```
{
  "name": "Ноутбук",
  "price": 999,
  "categoryId": 1
}
```

req.params
Параметры маршрута (URL)

```
/api/products/123
```

```
req.prrams.id = "123"
```

req.query
Строка запроса (Query String)

```
/api/products?page=1&limit=10
```

```
req.query = {
  "page": "1",
  "limit": "10"
}
```

2. Цепочки валидации (express-validator)

express-validator
Проверяем, валидируем и санитизируем данные

Валидация тела запроса (body)

```
body('name')
  .isString().notEmpty().withMessage('Название обязательно')

body('price')
  .isFloat({ min: 0 }).withMessage('Цена должна быть > 0')

body('categoryId')
  .isInt({ min: 1 }).withMessage('Некорректная категория')
```

Валидация параметров (param)

```
param('id')
  .isInt({ min: 1 }).withMessage('Некорректный ID')
```

Валидация строки запроса (query)

```
query('page')
  .optional().isInt({ min: 1 }).withMessage('page > 0')

query('limit')
  .optional().isInt({ min: 1, max: 100 }).withMessage('limit 1-100')
```

3. Проверка результата

```
import { validationResult }
from 'express-validator';

app.post('/api/products',
[
  // цепочки валидации
  body('name').isString().notEmpty(),
  body('price').isFloat({ min: 0 }),
  param('id').isInt({ min: 1 }),
  query('page').optional().isInt(),
], async (req, res) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    return res.status(400).json({
      message: 'Validation failed',
      errors: errors.array()
    });
  }

  // данные валидны — продолжаем
  next();
});
```

4a. Успешная валидация

errors.isEmpty() === true
Данные прошли все проверки.

Дальше обработчик

- req.body (валидные данные)
- req.params (валидные данные)
- req.query (валидные данные)

Продолжаем выполнение **(next())**

4b. Ошибки валидации

errors.isEmpty() === false
Возвращаем 400 Bad Request с описанием ошибок.

```
HTTP/1.1 400 Bad Request
Content-Type: application/json
{
  "message": "Validation failed",
  "errors": [
    {
      "type": "field",
      "msg": "Название обязательно",
      "path": "name",
      "location": "body"
    }
  ]
}
```

Преимущества такого подхода

- Единый способ валидации для body, params и query.
- Подробные и понятные сообщения об ошибках.
- Можно выполнять санитизацию данных.
- Легкая интеграция в middleware цепочку.

Важно помнить

- Валидация на сервере обязательна (client-side — только для удобства).
- Проверяйте только необходимые поля (allowed list).
- Используйте понятные сообщения об ошибках.
- Не передавайте дальше необработанные данные.

Результат

Надёжная проверка входящих данных, меньше ошибок и более безопасное API.

Правила для создания Product

```javascript
import { body } from "express-validator";

export const productRules = [
  body("name")
    .isString().withMessage("Название должно быть строкой")
    .isLength({ min: 2, max: 150 }),

  body("price")
    .isInt({ min: 1 }).withMessage("Цена должна быть положительной")
    .toInt()
];
```

Что важно

- Каждое поле проверяется собственной цепочкой.
- withMessage() задаёт сообщение пользователю.
- toInt() преобразует прошедшее проверку значение к числу.
- Это учебный минимальный набор; другие поля добавляются аналогично.

Что такое middleware в этой задаче

- Middleware — функция Express, которая выполняется между получением запроса и контроллером.
- Проверки productRules тоже работают как middleware.
- Если данных недостаточно или они неверны, вернём HTTP 400.
- Если данные подходят, вызываем next() и передаём управление дальше.

Собираем ошибки: validationResult()

```javascript
import { validationResult } from "express-validator";

export function validateRequest(req, res, next) {
  const errors = validationResult(req);

  if (!errors.isEmpty()) {
    return res.status(400).json({
      message: "Ошибка в данных",
      errors: errors.array()
    });
  }

  next();
}
```

Что важно

- validationResult(req) получает ошибки, накопленные правилами.
- HTTP 400 Bad Request означает некорректный запрос.
- return завершает обработку ошибочного запроса.
- next() передаёт корректный запрос следующей функции.

Подключаем проверки к маршруту

```javascript
router.post(
  "/",
  requireAuth,
  requireRole("admin"),
  productRules,
  validateRequest,
  createProduct
);
```

Что важно

- Сначала проверяется JWT, затем роль admin.
- Затем express-validator проверяет данные товара.
- Контроллер createProduct выполняется только после успешных проверок.

Проверяем ID товара в URL

```javascript
import { param } from "express-validator";

export const idRules = [
  param("id")
    .isInt({ min: 1 })
    .withMessage("ID должен быть положительным целым числом")
    .toInt()
];
```

Что важно

- param() — это функция библиотеки express-validator, которая используется для проверки параметров маршрута, переданных в URL (req.params).
- Параметр id приходит из URL как строка.
- Проверка не позволяет передать abc или отрицательное число.
- После idRules подключается тот же validateRequest.

Учебный вопрос 3. Безопасно передаём данные в Prisma

- Разберём, почему недостаточно проверить только отдельные поля, если весь req.body уходит в базу.

Что такое mass assignment

- Mass assignment — риск, когда сервер без отбора передаёт в обновление объекта все поля, присланные клиентом.
- Например, при регистрации клиент может дополнительно прислать role: "admin".
- Если сервер использует data: req.body без ограничений, могут измениться служебные поля.
- Правильный подход — явно выбирать поля, которые пользователю разрешено задавать.

Сравнение двух подходов

Опасно	Безопаснее
prisma.products.create({ data: req.body })	prisma.products.create({ data: req.validated })
Передаются все полученные поля	Передаются только разрешённые и проверенные
Могут попасть неожиданные свойства	Состав данных определяется правилами

Что такое matchedData()

- matchedData(req) — функция express-validator, извлекающая значения полей, охваченных правилами проверки.
- Она помогает не передавать в Prisma дополнительные поля из необработанного req.body.
- Вызывать её нужно после проверки ошибок.
- Список правил должен включать ВСЕ поля, которые действительно нужны контроллеру.

Дополняем validateRequest: matchedData()

```javascript
import { validationResult, matchedData }
  from "express-validator";

export function validateRequest(req, res, next) {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    return res.status(400).json({
      message: "Ошибка в данных",
      errors: errors.array()
    });
  }
  req.validated = matchedData(req);
  next();
}
```

Что важно

- matchedData(req) создаст объект только с полями, для которых были выполнены успешные правила валидации.
- req.validated — добавленное нами свойство объекта запроса; это не встроенное поле Express.
- Далее Controller читает req.validated вместо req.body.

Контроллер использует только проверенные поля

```javascript
export async function createProduct(req, res, next) {
  try {
    const product = await prisma.products.create({
      data: req.validated
    });
    res.status(201).json(product);
  } catch (error) {
    next(error);
  }
}
```

Что важно

- Это упрощённый пример: предполагается настроенный Prisma client.
- Неожиданное поле role не попадёт в req.validated, если для него нет правила.
- В рабочем проекте создание товара можно передавать отдельному Service.

Raw request → validators → matchedData → Controller → Service

Проверяем входные данные. В сервис и базу попадают только разрешённые поля.

Raw request

Входящий HTTP-запрос от клиента (`req.body`)

```
POST /api/products
Content-Type: application/json

{
  "name": "USB Hub",
  "price": 12000,
  "categoryId": 1,
  "inStock": true,
  "role": "admin"
}
```

Клиент может прислать любые поля, в том числе неожиданные.

Validators

express-validator

Проверяем разрешённые поля и запрещаем неожиданные.

```
const { body, checkExact } = require('express-validator')
const productValidators = [
  body('name').isString()
    .notEmpty(),
  body('price').isInt({ min: 0 }),
  body('categoryId').isInt(),
  body('inStock').isBoolean(),
  checkExact() // только эти поля
]
```

- Проверяем: name, price, categoryId, inStock
- Не принимаем неизвестные поля (checkExact / allowlist)

`"role": "admin"`

Неожиданное поле

Отклоняется и не попадает дальше в обработку.

matchedData(req)

Получаем только проверенные поля из запроса.

```
req.validated = {
  "name": "USB Hub",
  "price": 12000,
  "categoryId": 1,
  "inStock": true
}
```

Только проверенные поля.

Неожиданные поля (например, role) не попадают в req.validated.

Controller

Controller использует `req.validated`, а не `req.body`.

```
async function
createProduct(req, res) {
  const data = req.validated

  const product =
    await productService.
    create(data)

  res.status(201).json({
    id: product.id
  })
}
```

Service

В Service и Prisma не попадают неожиданные поля.

```
async function create(data) {
  return prisma.product.create({
    data: {
      name:  data.name,
      price: data.price,
      categoryId: data.categoryId,
      inStock: data.inStock
    }
  })
}
```

Prisma

В базу данных попадают только разрешённые поля.

Клиент не должен протащить `role: "admin"` в Prisma

Не доверяйте данным от клиента.
Всегда валидируйте и используйте только проверенные поля (`req.validated` / `matchedData`).

Что не умеет express-validator

- Не проверяет автоматически, существует ли categoryId в PostgreSQL.
- Не решает, хватает ли товара на складе.
- Не определяет права пользователя на изменение конкретной записи.
- Для этого нужны Service, Prisma и серверная авторизация.
- Валидация — только один из защитных слоёв.

Учебный вопрос 4. Ограничение запросов и Helmet

- Теперь защитим приложение не только от неверных значений, но и от некоторых нежелательных особенностей HTTP.

Что значит HTTP hardening

- Hardening — усиление безопасности приложения с помощью дополнительных настроек.
- Пример 1: ограничить размер JSON, чтобы не обрабатывать огромные тела запросов.
- Пример 2: добавить защитные HTTP-заголовки в ответы сервера.
- Эти настройки дополняют JWT, права и валидацию, а не заменяют их.

Зачем ограничивать размер JSON

- Даже корректный JSON может оказаться слишком большим.
- На разбор большого тела запроса расходуются ресурсы сервера.
- У большинства обычных запросов Internet Shop небольшой размер.
- Поэтому можно задать разумный лимит и отклонять слишком крупные тела.

Настраиваем лимит тела запроса

```javascript
app.use(express.json({ limit: "100kb" }));
```

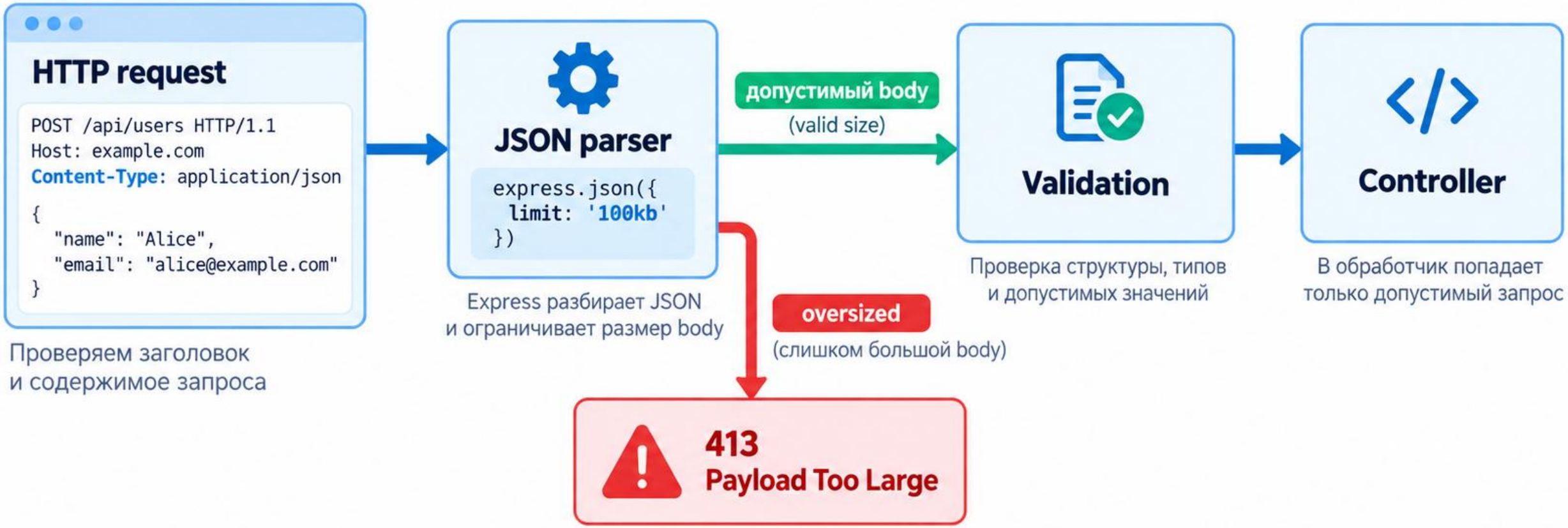
Что важно

- 100kb — пример учебного ограничения для JSON.
- При превышении установленного размера парсер отклоняет запрос.
- Клиент может получить HTTP 413 Content Too Large.
- Подбирайте лимит с учётом реальных размеров запросов приложения.

Request hardening: Content-Type + size limit + validation

Express.js REST API — безопасная обработка входящих запросов

Express.js



Идея:

Слишком большой request отклоняется раньше.
Допустимый JSON проходит дальше: **parser → validation → controller.**

Безопасные API — надёжные приложения

Что такое HTTP-заголовки ответа

- HTTP response headers — служебные данные, которые сервер отправляет браузеру вместе с ответом.
- Они сообщают браузеру правила обработки полученного содержимого.
- Например, можно запретить угадывание MIME-типа или ограничить загрузку скриптов.
- Часть таких заголовков полезна для безопасности frontend.

Что такое Helmet

- Helmet — middleware для Express, устанавливающее набор защитных HTTP-заголовков ответа.
- Оно помогает настроить браузерные ограничения без ручного перечисления каждого заголовка.
- Например, Helmet задаёт Content-Security-Policy и X-Content-Type-Options.
- Helmet не шифрует данные и не проверяет JWT или содержимое JSON.

Зачем нужен Helmet в Internet Shop

- Наш Express не только отдаёт JSON API, но и обслуживает HTML/frontend.
- Защитные заголовки браузер использует при обработке этой страницы.
- Helmet снижает риск некоторых браузерных атак и небезопасных способов обработки содержимого.
- Его настройки нужно проверить с текущими HTML, JavaScript и локальной разработкой.

Установка Helmet

```bash
npm install helmet
```

Что важно

- Устанавливаем пакет через npm в каталоге backend-проекта.
- Затем импортируем его в серверный app.js.

Подключение Helmet в Express

```javascript
import helmet from "helmet";

app.use(helmet());
```

Что важно

- app.use(helmet()) подключает middleware.
- Разместите его до маршрутов приложения.
- Проверьте заголовки ответа в DevTools → Network → Headers.

Основные заголовки Helmet простыми словами

Заголовок	Что помогает ограничивать
Content-Security-Policy (CSP) (политика безопасности содержимого страницы.)	Откуда браузеру можно загружать скрипты и другие ресурсы
X-Content-Type-Options: nosniff	Запрет угадывания типа содержимого
Referrer-Policy	Какие сведения об адресе страницы передаются по ссылке
Strict-Transport-Security (HSTS)	Использование HTTPS для хоста после получения политики

Важный нюанс: Helmet и localhost

- Настройки Helmet по умолчанию включают строгие правила CSP.
- При локальной разработке браузер может попытаться преобразовать HTTP в HTTPS из-за upgrade-insecure-requests.
- Если frontend перестал загружаться после подключения Helmet, проверьте Console и CSP.
- Для учебного HTTP-окружения при необходимости отключают только upgrade-insecure-requests, а не весь Helmet.

Пример Helmet для локального HTTP

```javascript
import helmet from "helmet";

app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      "upgrade-insecure-requests": null
    }
  },
  strictTransportSecurity: false
}));
```

Что важно

- Учебная настройка только для локального HTTP.
- В production с HTTPS политику безопасности нужно настроить отдельно.
- Не копируйте эту конфигурацию как готовый вариант для публичного сервера.

Учебный вопрос 5. Rate limiting и защита от частых запросов

- Рассмотрим, что происходит, когда клиент слишком часто обращается к одному API.

Что такое rate limiting

- Rate limiting — ограничение количества запросов, которое клиент может выполнить за заданный период.
- Пример: не больше 10 попыток входа за 15 минут для определённого клиентского ключа.
- Если лимит превышен, сервер обычно отвечает HTTP 429 Too Many Requests.
- Это помогает сдерживать слишком частые автоматические обращения.

Зачем ограничивать запросы к login

- Злоумышленник может автоматически пробовать много паролей подряд.
- Более строгий лимит для /api/auth/login усложняет массовые попытки входа.
- Для публичного чтения товаров можно выбрать другой, более мягкий лимит.
- Rate limiting не заменяет проверку пароля, JWT и авторизацию.

Установка express-rate-limit

```bash
npm install express-rate-limit
```

Что важно

- Пакет позволяет подключить ограничение частоты как middleware.
- Это не часть express-validator или Helmet — отдельная задача.

Простой лимит для всего API

JAVASCRIPT

```
import { rateLimit } from "express-rate-limit";

const apiLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  limit: 300,
  standardHeaders: "draft-8",
  legacyHeaders: false
});

app.use("/api", apiLimiter);
```

Что важно

- windowMs — время подсчёта запросов: здесь 15 минут.
- limit — максимум запросов за это окно для клиентского ключа.
- После превышения лимита middleware возвращает HTTP 429.
- Лимит 300 приведён для примера; он не универсален.

Rate limiting в Express.js: защита от слишком частых запросов

Обычные пользователи получают доступ к API. При превышении лимита запросов возвращается 429 Too Many Requests.
Для чувствительных маршрутов (например, /auth/login) используется более стргий лимитер.

Internet Shop
Learn. Build. Deploy.

1. Обычный трафик — доступ разрешён

Пользователь 1 — Запрос
Пользователь 2 — Запрос
Пользователь 3 — Запрос
...
Пользователь N — Запрос

Rate Limiter (общий)

В пределах лимита

Например:
- 100 запросов
- за 15 минут на один IP

OK

API

```
GET   /api/products
GET   /api/orders
POST  /api/orders
GET   /api/profile
...
```

Запрос обработан (200 OK)

2. Слишком много запросов — блокировка

Бот / Скрипт
Злоумышленник
Автоматические запросы

Rate Limiter (общий)

Лимит превышен

> 100 запросов за 15 минут с одного IP

429

Ответ сервера

```
HTTP/1.1 429 Too Many Requests
Content-Type: application/json
Retry-After: 900
{
  "error": "Too Many Requests",
  "message": "Rate limit exceeded.",
  "retryAfter": 900
}
```

Браузер/клиент должен подождать
Повторные запросы можно отправлять через указанное время (Retry-After).

3. Более строгий лимитер для /auth/login

Страница входа

user@example.com
••••••••
Войти

POST /auth/login
Много попыток

Rate Limiter (строгий для логина)

Жёсткие ограничения

Например:
- 5 запросов за 15 минут на один IP

429

Ответ сервера

```
HTTP/1.1 429 Too Many Requests
Content-Type: application/json
Retry-After: 900
{
  "error": "Too Many Login Attempts",
  "message": "Please try again later."
}
```

Дополнительные меры для /auth/login

- Строгий rate limit (меньше запросов).
- Постепенная блокировка (экспоненциальная задержка).
- CAPTCHA после нескольких неудачных попыток.
- Логирование подозрительной активности.
- Возможность временной блокировки IP.

Пример настройки (express-rate-limit)

```
import rateLimit from 'express-rate-limit';
// Общий лимитер
export const apiLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,   // 15 минут
  max: 100,                   // 100 запросов
  standardHeaders: true,
  legacyHeaders: false,
  message: { error: 'Too Many Requests' }
});
```

Строгий лимитер для /auth/login

```
export const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,   // 15 минут
  max: 5,                     // 5 попыток
  standardHeaders: true,
  legacyHeaders: false,
  message: { error: 'Too Many Login Attempts' }
});
```

Применение в маршрутах

```
import { apiLimiter, loginLimiter } from './middleware/limit.js';

app.use('/api', apiLimiter);
app.post('/auth/login', loginLimiter, loginController);
```

Результат

- API защищён от злоупотреблений.
- Стабильная работа сервера.
- Снижение нагрузки.
- Более безопасная авторизация.
- Лучший пользовательский опыт для легитимных пользователей.

Более строгий лимит для входа

```javascript
const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  limit: 10,
  standardHeaders: "draft-8",
  legacyHeaders: false
});

router.post("/login", loginLimiter, loginHandler);
```

Что важно

- У логина отдельный лимит 10 попыток за 15 минут — только пример.
- loginHandler — существующий обработчик входа в вашем проекте.
- Если уже есть loginValidation и validateRequest, оставьте их в цепочке.
- Следите, чтобы общий и отдельный лимиты не конфликтовали с тестовыми сценариями.

Ограничения rate limiting

- В учебном приложении со стандартным хранилищем счётчики могут храниться в памяти процесса.
- В нескольких экземплярах сервера требуется общее хранилище счётчиков.
- За reverse proxy нужно корректно определить адрес клиента.
- Rate limiting может создавать неудобства реальным пользователям при слишком строгих лимитах.
- Это не полноценная защита от DDoS.

Что такое defence in depth

- Defence in depth — многоуровневая защита: несколько независимых проверок уменьшают риск ошибки.
- JWT отвечает на вопрос «кто отправил запрос?», requireRole — «что ему разрешено?».
- express-validator проверяет форму данных; Service проверяет бизнес-правила.
- Helmet задаёт HTTP-заголовки, rate limiting сдерживает частые запросы.
- Каждый слой решает собственную задачу.

Security middleware pipeline в Express.js (REST API)

Последовательная обработка запроса: от входящего HTTP-запроса до ответа. На каждом этапе возможна остановка с соответствующим кодом ошибки.

Internet Shop
Learn. Build. Deploy.

1. Request

Клиент
(Browser / API client)

```
GET /api/orders
Authorization: Bearer ...
Content-Type: application/json
Origin: http://localhost:5500
{ ... }
```

2. CORS

CORS
(доступ с других origin)

- Проверка Origin
- Разрешённые методы и заголовки
- Credentials (при необходимости)

3. Helmet

Helmet
(security headers)

- X-Content-Type-Options
- X-Frame-Options
- Content-Security-Policy
- Referrer-Policy
- и другие заголовки безопасности

4. Body limit

JSON parser
(limit 100kb)

- `express.json({ limit: "100kb" })`
- Проверка Content-Type: application/json

5. Rate limiter

Rate limiter
(защита от брута и DDoS)

- Общий лимит (например, 100/15min)
- Строгий лимит для `/auth/login`

6. requireAuth

requireAuth
(проверка токена)

- Проверка JWT
- Валидность токена
- Загрузка пользователя (req.user)

7. requireRole

requireRole
(проверка роли)

- Нровимер: `'user'`, `'admin'`
- Проверка прав доступа к ресурсу

8. Validation

Validation
(проверка данных)

- express-validator
- Проверка body/params/query
- Санитизация
- Проверка только разрешённых полей (allowlist)

9. matchedData

matchedData
(чистые данные)

- Извлекаем только валидные поля
- Игнорируем неожиданные поля
- `req.validated = matchedData(req)`

10. Controller

Controller
(обработка запроса)

- Бизнес-логика эндпоинта
- Вызов сервисов
- Формирование ответа

11. Service

Service
(бизнес-логика)

- Работа с БД через Prisma
- Дополнительные проверки
- Транзакции (при необходимости)

12. Prisma

Prisma
(ORM)

- Проверенные данные
- Безопасные запросы (параметризованные)

13. PostgreSQL

PostgreSQL
(база данных)

14. Response

Response
(JSON ответ)

```
HTTP/1.1 200 OK
{
  "success": true,
  "data": { ... }
}
```

CORS Denied

Origin не разрешён

```
HTTP/1.1 403 Forbidden
{
  "error": "CORS denied"
}
```

413 Payload Too Large

Размер тела запроса превышает лимит (> 100kb)

```
HTTP/1.1 413 Payload Too Large
{
  "error": "Request body too large"
}
```

429 Too Many Requests

Превышен лимит запросов

```
HTTP/1.1 429 Too Many Requests
{
  "error": "Too many requests",
  "retryAfter": 900
}
```

401 Unauthorized

Нет авторизации или неверный токен

```
HTTP/1.1 401 Unauthorized
{
  "error": "Unauthorized",
  "message": "Invalid or missing token"
}
```

403 Forbidden

Недостаточно прав (роль не подходит)

```
HTTP/1.1 403 Forbidden
{
  "error": "Forbidden",
  "message": "Insufficient permissions"
}
```

400 Bad Request

Ошибка валидации данных

```
HTTP/1.1 400 Bad Request
{
  "error": "Validation failed",
  "details": [
    { "field": "email",
      "msg": "Invalid email" }
  ]
}
```

Порядок middleware в Express.js

1. **cors()** — разрешение кросс-доменных запросов
2. **helmet()** — security headers
3. express.json({ limit: "100kb" }) — ограничение размера тела
4. rateLimit() — ограничение количества запросов
5. requireAuth() — проверка токена
6. requireRole() — проверка роли
7. validation chains — проверка входящих данных
8. matchedData() — оставляем только валидные поля
9. controller → service → prisma → database

Преимущества такого подхода

- Многоуровневая защита (defense in depth)
- Ранняя остановка вредоносных запросов
- Чистые и проверенные данные в контроллере
- Защита от брута, DDoS и больших payload
- Контроль доступа на основе ролей
- Минимизация риска инъекций и утечки данных
- Более стабильное и безопасное API

Рекомендации

- Размещайте middleware в правильном порядке
- Используйте allowlist валидацию (matchedData)
- Устанавливайте разумные лимиты (size, rate)
- Применяйте строгие правила для /auth/login
- Логируйте ошибки и подозрительную активность
- Используйте HTTPS в production
- Регулярно обновляйте зависимости

Результат

Безопасное, надёжное и предсказуемое REST API

```
HTTP/1.1 200 OK
{
  "success": true,
  "data": { "id": 1, "name": "Order" }
}
```

Какой инструмент какую проблему решает

Проблема	Инструмент	Ответ при нарушении
Отрицательная цена	express-validator	400
Нет JWT	requireAuth	401
Недостаточно прав	requireRole	403
Слишком большой JSON	express.json({ limit })	413
Слишком частые запросы	express-rate-limit	429
Защитные заголовки ответа	Helmet	Добавляет HTTP headers

Практика: что добавить в Internet Shop

- 1. Правила проверки Product: name и price.
- 2. Общий validateRequest с HTTP 400; передавать в Prisma только разрешённые поля.
- 3. Проверку :id в URL.
- 4. Ограничение размера JSON и Helmet.
- 5. Rate limiting для /api и более строгий — для /api/auth/login.

Как проверить результат

Проверка	Ожидаемое поведение
POST Product с price = -1	HTTP 400, товар не создан
POST Product без JWT	HTTP 401
POST Product с JWT роли user	HTTP 403
Очень большой JSON	HTTP 413
Слишком много запросов к login	HTTP 429
GET /api/products	Обычный запрос продолжает работать

Типичные ошибки при внедрении

- Если middleware подключён после маршрута, он может не повлиять на этот маршрут.
- Если забыть validateRequest, правила накопят ошибки, но контроллер всё равно может выполниться.
- Если отправить req.body целиком в Prisma, неожиданные поля всё ещё опасны.
- Если после Helmet пропал frontend, проверьте Console и защитные заголовки.
- Если лимит слишком строгий, обычные пользователи начнут получать 429.

Контрольные вопросы

1. Что такое серверная валидация и почему frontend-проверки недостаточно?
2. Для чего предназначен express-validator?
3. Что делают body(), param() и validationResult()?
4. Зачем нужен matchedData() и чем опасен data: req.body?
5. Какую проблему решает лимит размера JSON?
6. Что такое Helmet и какие заголовки он добавляет?
7. Для чего нужен rate limiting и что означает 429?
8. Почему ни один из этих инструментов не заменяет JWT и авторизацию?

Итоги занятия

1. Пользовательские данные нельзя считать доверенными — сервер обязан их проверять.
2. express-validator описывает правила проверки; validationResult() помогает вернуть ошибку 400.
3. matchedData() позволяет передать в Service/Prisma только предусмотренные поля.
4. Ограничение JSON-body защищает от чрезмерно больших запросов; возможен ответ 413.
5. Helmet добавляет защитные заголовки HTTP-ответа.
6. Rate limiting ограничивает частоту обращений; при превышении возвращается 429.
7. Все меры дополняют JWT, роли, бизнес-правила и ограничения PostgreSQL.